School of Computer Science and Engineering

BCSE307L - Compiler Design

Question Bank

Module 3

1. Construct a syntax directed translation scheme that translates arithmetic expressions from postfix notation into infix notation. Give annotated parse tree for the expressions: ***95-2**** and ***952*-.***

2. Construct a Syntax-Directed Definition to perform type checking and type conversion for arithmetic expressions. Show the annotated parse tree for: a + b * c, where a is int, b is float, and c is int.

3. Design a Syntax-Directed Definition (SDD) to translate postfix arithmetic expressions into prefix notation. Draw the annotated parse tree and show the translation for: 95+2*.

4. Construct a Syntax-Directed Translation Scheme to generate **three-address code (TAC)** for arithmetic expressions. Give the annotated parse tree and generate three-address code for:

a. a + b * c

b. (a + b) * (c - d)

5. Using the following grammar, write a SDT to count the number of reductions required to parse the arithmetic expression id + id * id. Show the complete sequence of reductions.

E → E + T | T

T → T * F | F

F → (E) | id

6. Consider the following grammar for Boolean expressions and if-else statements:

S → if B then S1 else S2

S → assign id = E

B → E = = E | E > E

E → E + T | E – T

T → T*F | T/F | F

F → (E) | num

Write a SDD that evaluates arithmetic expressions. Evaluates Boolean expressions(B). Given the below input, construct the annotated parse tree showing how your SDD evaluates the Boolean expression and simulates the conditional assignment: if (3 + 4) > 5 then assign x = 10 else assign x = 20

7. The following grammar generates expressions formed by applying an arithmetic operator + to integer and real constants. When two integers are added, the resulting type is integer, otherwise, it is real

E → E+T | T

T → num.num | num

a. Give a syntax-directed definition to determine the type of each sub expression.
b. Extend the syntax-directed definition of (a) to translate expressions into postfix notation as well as determining types. Use the unary operator 'inttoreal' to convert an integer value into an equivalent real value, so that both operands of + in the postfix form have the same type.

8. Formulate SDT for syntax tree construction for a simple desk calculator and show how the expression 2+4*5^2 is evaluated.

9. Consider the syntax directed translation given by the following grammar and semantic rules. Here N, I, F and B are non-terminals. N is the starting non-terminal, and #,0 and 1 are lexical tokens corresponding to input letters "#", "0" and "1", respectively. X.val denotes the synthesized attribute (a numeric value) associated with a non-terminal X. I1 and F1 denote occurrences of I and F on the right-hand side of a production, respectively. For the tokens 0 and 1, 0.val=0 and 1.val=1. Find the value computed by the translation scheme and Draw an annotated parse tree showing the detailed computation of attribute values for the given string 10#011.

N→I#F {N.val=I.val+F.val}

I→I1B {I.val=(2 I1.val)+ B.val }

I→B {I.val= B.val }

F→BF1 {F.val=1/2 (B.val+F1.val)}

F→B {F.val =1/2 B.val}

B→0 {B.val=0.val}

B→1 {B.val=1.val}

10. We have the following grammar with translation rules. Here S, T, and R are Non-terminals; id represents an integer; id.val gives its integer value; # and % are operators.

S → S1 # T	{_____}
S → T	{S.val = T.val}
T → T1 % R	{_____}
T → R	{T.val = R.val}
R → id	{R.val = id.val}

When we evaluate an expression 20 # 10 % 5 # 8 % 2 % 2 using this translation scheme, the S.val is 80. Hence answer the following.

a. Fill the blanks with appropriate semantic rules.
b. Write about operator precedence and associativity.
c. Build a parse tree using productions.
d. Perform bottom-up evaluation using semantic rules.

11. Construct a Syntax-Directed Definition (SDD) for the following Grammar using synthesized attributes to calculate the decimal value of a binary number. For example, the translation of Binary number 111.001 should be the decimal number 7.125.

X → Y.Y | Y
Y → YB | B
B → 0 | 1

Module 4

1. Generate the Quadruple, Triple and Indirect Triple representation of the three-address code of the following statement a + b * c / e ↑ f + b * a

2. Generate 3AC for the following code snippet and represent the following code into Quadruple, triples and indirect triples form.

```
int main()
{
int i;
int a[10];
for (i = 0; i < 10; i++) {
a[i] = 0;
}
```

```
return 0;
}
```

3. Write SDT to generate an intermediate code for the following code snippets. Do you think back patching is required for the above case? Justify your answer with a detailed explanation.

a.
```
while (a < b && c != d)
{ count++; }
count+=4;
```

b.
```
if ((a < b && c != d && e <= f) || (g > h && i == j) || k != l)
   x = a + 1;
else
   x = b + 1;
```

c.
```
if (((a < b && c != d) || (e <= f && g > h)) && ((i == j || k != l) && m >= n))
```

4. Consider the following C like code that involves pointers and dynamic memory allocation.

```
int* allocateAndFillArray(int size)
{
int* arr = (int*) malloc(size * sizeof(int));
for (int i = 0; i < size; i++)
{
arr[i] = i * 2;
}
return arr;
}
int main()
{ int n = 5;
int* p = allocateAndFillArray(n);
int sum = 0;
for (int i = 0; i < n; i++)
{ sum += p[i];
}
return sum;
}
```

Generate the three-address code for the above code. Assume that the TAC has an instruction alloc(n) that performs the malloc().

5. Consider the following C code, that uses a switch-case statement to compute different operations based on an input value.

```
int result;
int x = 5;
switch(x)
{ case 1:
result = x + 1;
break;
case 2:
result = x * 2;
break;
case 3:
result = x - 3;
break;
default:
result = x;
}
```

Write semantic actions to generate intermediate code for the above switch-case statement. Show the annotated parse tree for the switch case statement.

6. Define Backpatching semantic actions the following Grammar:

E → E1 OR M E2
| E1 AND M E2
| not E1
| (E1)
| id1 relop id2
| true
| False
M → ε

Based on the translation scheme, generate intermediate code representation for the following expression.

a) a==b AND (c==d OR e==f)
b) (a==b AND c==d) AND e==f

7. Illustrate the steps of generating the intermediate code with backpatching. Include the backpatching lists and the final generated code.

Given the input code:

```
if ((a>b)&&(a>c)&&(a>d))
{printf(" largest is %d",a);}

else if ((b>a)&&(b>c)&&(b>d))
{printf(" largest is %d",b);}

else if ((c>a)&&(c>b)&&(c>d))
{printf(" largest is %d",c);}

else {printf("largest is %d",d);}
```

8. Consider the array declaration format: T a[d1][d2][d3];

Where: T is a primitive data type (int, float, or char), d1, d2, and d3 are the dimension sizes.

(a) Complete the missing part of the grammar rule to represent the array declaration.

(b) Define the semantic rules required to compute the total storage size for the given array.

(c) Using a bottom-up parsing approach, demonstrate the step-by-step semantic evaluation and computation of the total size for a sample input declaration (e.g., int A[2][3][4];).

(d) Specify the attributes and its types used in the syntax-directed definition.

D → T id [N1] [N2] [N3];

{D.type = _____

D.d1= _____

D.d2 = _____

D.d3 = _____

D.elemSize = T.size

D.totalSize = _____}

T → int {T.type = int, T.size = 4}

T → float {T.type = float, T.size = 4}

T → char {T.type = char, T.size = 1}

N → num {N.val = num.lexval}

9. Consider the arithmetic expression: a + b * (c - d) + a

(a) Construct the parse tree, abstract syntax tree (AST), and DAG for the given expression.

(b) Explain how the DAG identifies the common subexpression, if any, in the given expression.

(c) Generate the corresponding three-address code (TAC) using temporary variables.

(d) State the number of temporary variables required.

Module 5

1. Consider the following C program, that transposes a 3 x 3 matrix and sums the diagonal elements:

```
for (int i = 0; i < 3; i++)
{
for (int j = 0; j < 3; j++)
{
transpose[j][i] = arr[i][j];
if (i == j)
{ sum = sum + transpose[i][j];
}
}
}
```

Convert the above code into three-address code and partition the same into basic blocks. Construct the flow graph for the basic blocks, showing control flow between them.

2. Consider the assembly code generated by a compiler. Apply peephole optimization to optimize the code.

```
1. MOV R1, R2
2. MOV R2, R1
3. ADD R3, #0
4. JMP L1
5. MOV R4, #5
6. L1: MUL R5, #1
7. MOV R6, #0
8. ADD R6, R7
9. CMP R8, R9
```

10. JNE L2
11. JMP L3
12. L2: JMP L3
13. L3: NOP

3. Identify the basic blocks and construct a flow graph for the below prime number counting code from a large given array. Also identify loops in the flow graph.

```
for (i=2; i<=n; i++)
a[i] = TRUE;
count = 0;
s = sqrt(n);
for (i=2; i<=s; i++)
if (a[i]) /* i has been found to be a prime */
{
count++;
for (j=2*i; j<=n; j = j+i)
a[j] = FALSE; /* no multiple of i is a prime */
}
```

4. Give the DAG representation of basic block for the following three address code and apply suitable peephole optimization techniques to generate optimized code.

(1) t1 := 4*i
(2) t2 := a [t1]
(3) t3 := 4*i
(4) t4 :=b [t3]
(5) t5 := t2*t4
(6) t6 := prod +t5
(7) prod := t6
(8) t7 := i+1
(9) t8 :=4*2
(10) if t8 <5 goto (4)
(11) i := t7
(12) if i<=20 goto (1)

5. Given the following code snippet, identify the basic blocks, construct a control flow graph (CFG).

```
int globalVar;

void process(int value) {
    int localVar;
    if (value < 0) {
        throw "Negative value error";
    } else {
        for (int i = 0; i < value; i++) {
            if (i % 2 == 0) {
                globalVar += i;
            } else {
                localVar += i;
            }
        }
    }
    print(globalVar);
    print(localVar);
}

int main() {
    try {
        process(5);
    } catch (const char* e) {
        print(e);
    } finally {
        print("Execution completed.");
    }
}
```

6. Translate the following code snippet into three address code. Identify the basic blocks and construct a flow graph for the same.

```
for (i = 2; i <= n; i++) {
a[i] = TRUE; }
count = 0;
s = sqrt(n);
for (i = 2; i <= s; i++) {
if (a[i] == TRUE) {
count++;
for (j = 2 * i; j <= n; j = j + i) {
a[j] = FALSE;
}
}
}
```

7. Find intermediate code, leaders, basic blocks and flow graph for the following codes.

a.

```
if (a < b)
   x = a + 1;
else
   x = b + 1;
```

b. switch (x)
{
   case 1:
      y = 10;
      break;

   case 2:
      y = 20;
      break;

   default:
      y = 0;
}
c. i = 1;
while (i <= 5) {
   sum = sum + i;
   i = i + 1;
}

8. Consider the following three-address code. Apply suitable optimization techniques to generate the optimized three-address code. Explain in detail about the optimization techniques used for each optimization.

(1) t1 := 4 * i
(2) t2 := a[t1]
(3) t3 := 4 * i
(4) t4 := b[t3]
(5) t5 := t2 * t4
(6) t6 := x + t5
(7) x := t6
(8) t7 := i + 1
(9) t8 := 4 * 2
(10) t9 := t8 + 0
(11) t10 := t9 * 1
(12) y := t10
(13) t11 := i * 2
(14) t12 := t11 * 2
(15) z := t12

(16) t13 := x
(17) w := t13
(18) t14 := unused + 5
(19) goto L1
(20) goto L1
L1:
(21) if i < 20 goto L2
(22) goto L3